

Network Traffic Pattern Analysis

Alessio Sacco, Marco Rapelli

Politecnico di Torino

1859



Why analyze network traffic

- **Analyze network behavior** over time. Understanding network traffic patterns is a critical skill in modern cybersecurity and network management tasks. This is key for capacity planning, anomaly detection, and optimization.
- Some applications:
 - Network traffic forecasting
 - Intrusion detection and mitigation
 - Dynamic bandwidth allocation
 - Performance tuning and anomaly alerts



What is network traffic

- **Network traffic** represents the data moving across a computer network at a given time. It includes all types of digital communication between devices, servers, and applications. Network traffic types:
 - Inbound traffic
 - Outbound traffic
 - Internal traffic
 - External Traffic
- Network traffic is composed of **data packets**, which are small units of data transmitted over a network. Each packet contains:
 - Source IP, Origin of the packet
 - Destination IP, Target of the packets
 - Protocol, Communication protocol (TCP, UDP)
 - Payload, Actual data being transmitted



Network Traffic Capture Tools

- To capture network traffic, it is possible to use **tcpdump**, a powerful command-line packet analyzer:

```
## Install tcpdump
sudo apt-get update
sudo apt-get install tcpdump

## Capture packets on eth0 interface
sudo tcpdump -i eth0

## Capture specific protocol traffic
sudo tcpdump -i eth0 tcp

## Save captured packets to a file
sudo tcpdump -i eth0 -w capture.pcap
```



Network traffic measurement metrics

Some examples of traffic measurement metrics (what we want to monitor):

- Bandwidth: Total data transferred
- Latency: Time taken for data transmission
- Packet Loss: Percentage of packets not reaching destination
- Throughput: Actual data successfully transmitted

Traffic pattern analysis

Traffic pattern analysis is a critical technique for identifying network behavior, detecting anomalies, preventing potential security threats, and opportunistically manage resources.

Some techniques work by **establishing a baseline**:

- Normal traffic measurement
- Peak usage times
- Typical protocol distribution (e.g., identify percentage of different protocols and detect unexpected protocol usage)
- Standard bandwidth consumption

Traffic pattern analysis (cont'd)

We can identify two categories of network traffic prediction:

- **Long-period prediction** (hours to years) to provide detailed forecasting of the workload and traffic patterns to estimate future capacity requirements and therefore allow for more accurate planning.
- **Short-period prediction** (milliseconds to minutes) is typically applicable for dynamic resource allocation (e.g., routing or network adaptation).



Practical Traffic Analysis with Python

- Using **scapy** library

```
import scapy.all as scapy
import pandas as pd

def analyze_network_traffic(interface, duration=60):
    packets = scapy.sniff(iface=interface, timeout=duration)

    ## Extract packet details
    packet_data = []
    for packet in packets:
        if packet.haslayer(scapy.IP):
            packet_info = {
                'Source IP': packet[scapy.IP].src,
                'Destination IP': packet[scapy.IP].dst,
                'Protocol': packet[scapy.IP].proto
            }
            packet_data.append(packet_info)

    return pd.DataFrame(packet_data)

## Usage example
traffic_df = analyze_network_traffic('eth0')
print(traffic_df)
```



Anomaly Detection



Anomaly detection Methods

■ Rule-Based Analysis

- Predefined rules for suspicious activities
- Immediate flagging

■ Statistical Analysis

- Compares current traffic against historical data
- Identifies deviations

■ Machine Learning

- Uses algorithms to predict normal behavior
- Adaptive detection



Advanced Threat Monitoring Script

- A rule-based example

Apply here your model



```
python

import socket
import logging
from scapy.all import *

class NetworkSecurityMonitor:
    def __init__(self, interface):
        self.interface = interface
        logging.basicConfig(filename='security_log.txt', level=logging.WARNING)

    def detect_suspicious_traffic(self, packet):
        ## Analyze packet characteristics
        if packet.haslayer(IP):
            src_ip = packet[IP].src
            dst_ip = packet[IP].dst

            ## Check for potential suspicious patterns
            if self._is_suspicious_connection(src_ip, dst_ip):
                self._log_security_event(packet)

    def _is_suspicious_connection(self, src_ip, dst_ip):
        ## Implement custom logic for suspicious connection detection
        suspicious_ips = ['192.168.1.100', '10.0.0.50']
        return src_ip in suspicious_ips or dst_ip in suspicious_ips

    def _log_security_event(self, packet):
        logging.warning(f"Suspicious Packet Detected: {packet.summary()}")

    def start_monitoring(self):
        print("Network Security Monitoring Started...")
        sniff(iface=self.interface, prn=self.detect_suspicious_traffic)

## Usage
monitor = NetworkSecurityMonitor('eth0')
monitor.start_monitoring()
```



DoS Attack Example

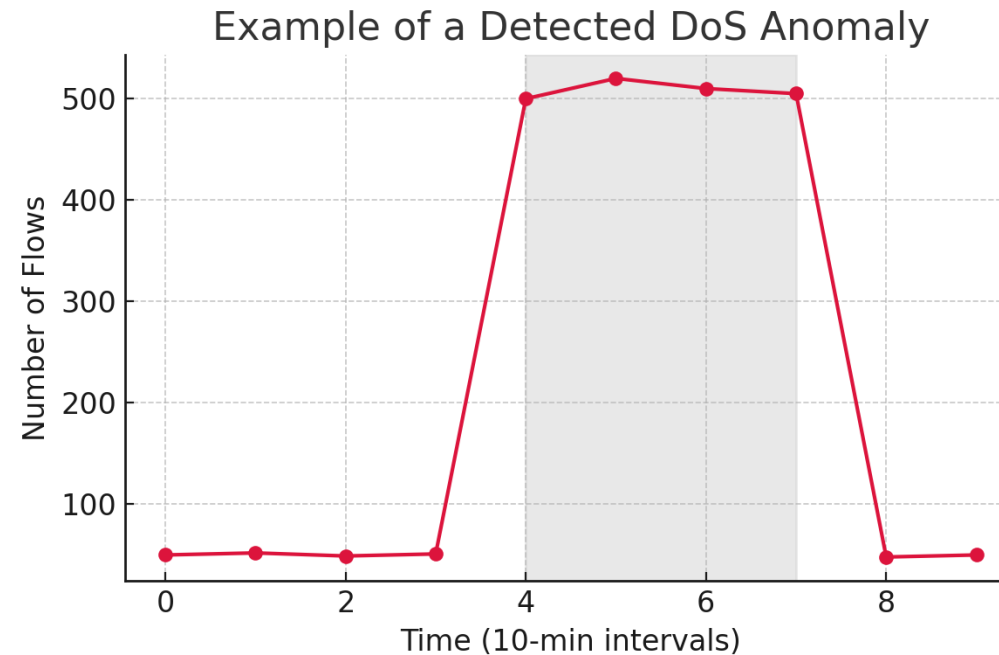
Detected anomaly from IP address ID xxx showed symptoms of Denial of Service (DoS):

- High number of flows and packets, but **low data size** (1-packet flows).
- Low variation in **destination ports and IPs** — consistent with attack behavior.
- All traffic was **incoming** (dir_ratio near 0), mostly **UDP**.
- **TTL** constant, suggesting single source.
- All metrics matched known (by AI/ML) DoS patterns



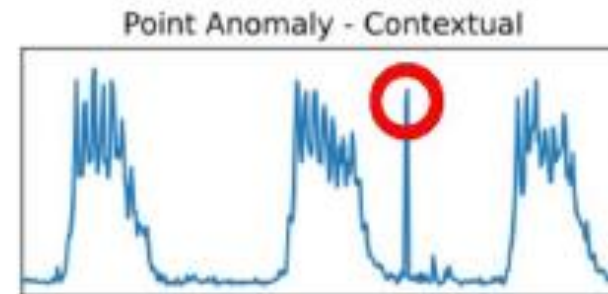
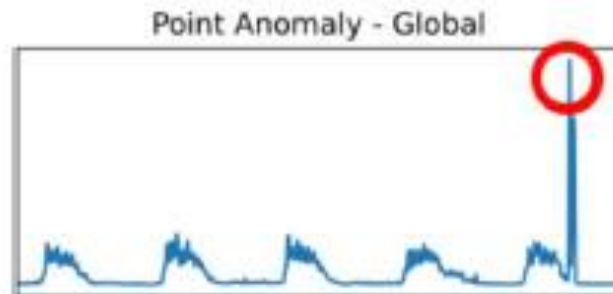
DoS Attack Example (cont'd)

The grey area highlights a sharp increase in traffic flows without a corresponding increase in bytes or destinations, consistent with a DoS attack pattern.



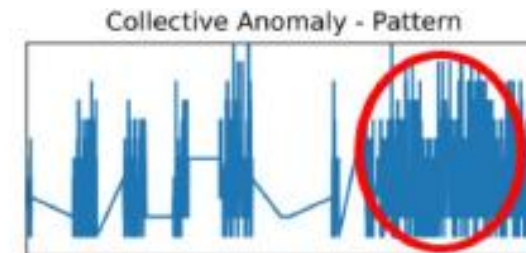
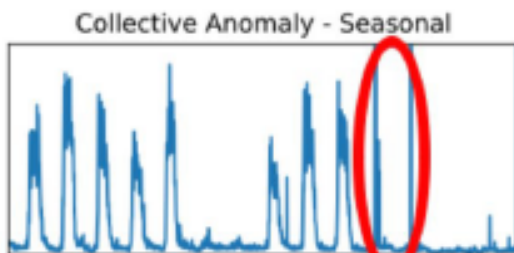
Types of anomalies

- **Point Anomaly**: a single data point that significantly deviates from the rest of the datapoints in the time series.
 - Example: Sudden spike in traffic due to a scan.
- There are two types of point anomalies:
 - **Global** - A global outlier is a data point that deviates significantly from the overall pattern or distribution of the entire dataset.
 - **Contextual** - A data point that is an outlier within a specific context or condition but not necessarily when viewed in a different context.



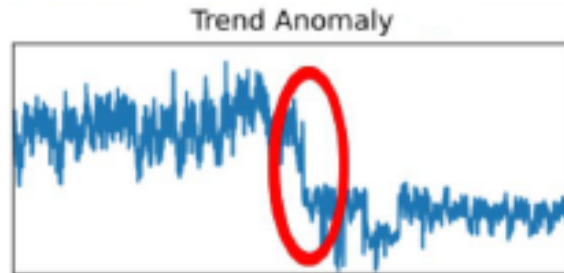
Types of anomalies

- **Collective Anomaly**: a sequence of datapoints that is anomalous when considered together, even if individual points might not be.
 - Example: High usage at night when usage is typically low.
- There are three types of collective anomalies:
 - **Seasonal** - A contiguous subsequence of datapoints that is anomalous compared to the seasonal pattern of the time series.
 - **Subsequence** - A contiguous subsequence of datapoints that is anomalous compared to the rest of the time series.
 - **Pattern** - A sequence of datapoints that together form an unusual pattern that does not conform to known time series patterns.



Types of anomalies

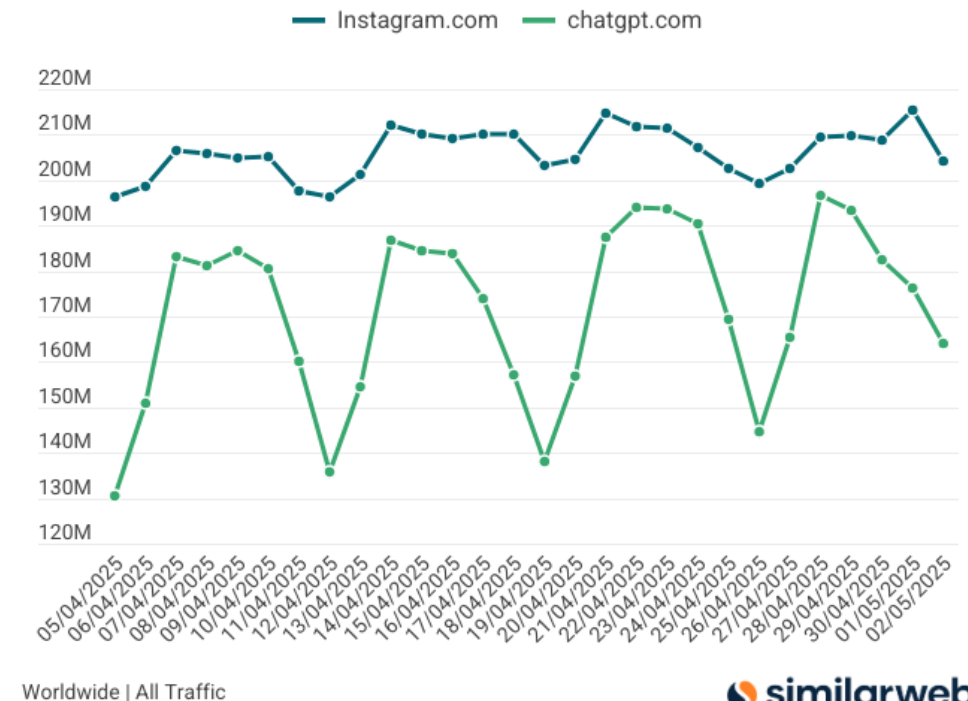
- **Trend Anomaly**: an unexpected change in the trend of the time series data, such as a sudden shift from a positive to a negative trend. Similar data behaviors in data science are called data or content drifts, and sometimes, this anomaly type is called a *Drift Anomaly*.
 - Example: Series of packets indicating a DDoS attack.



Be careful, trends are application-specific

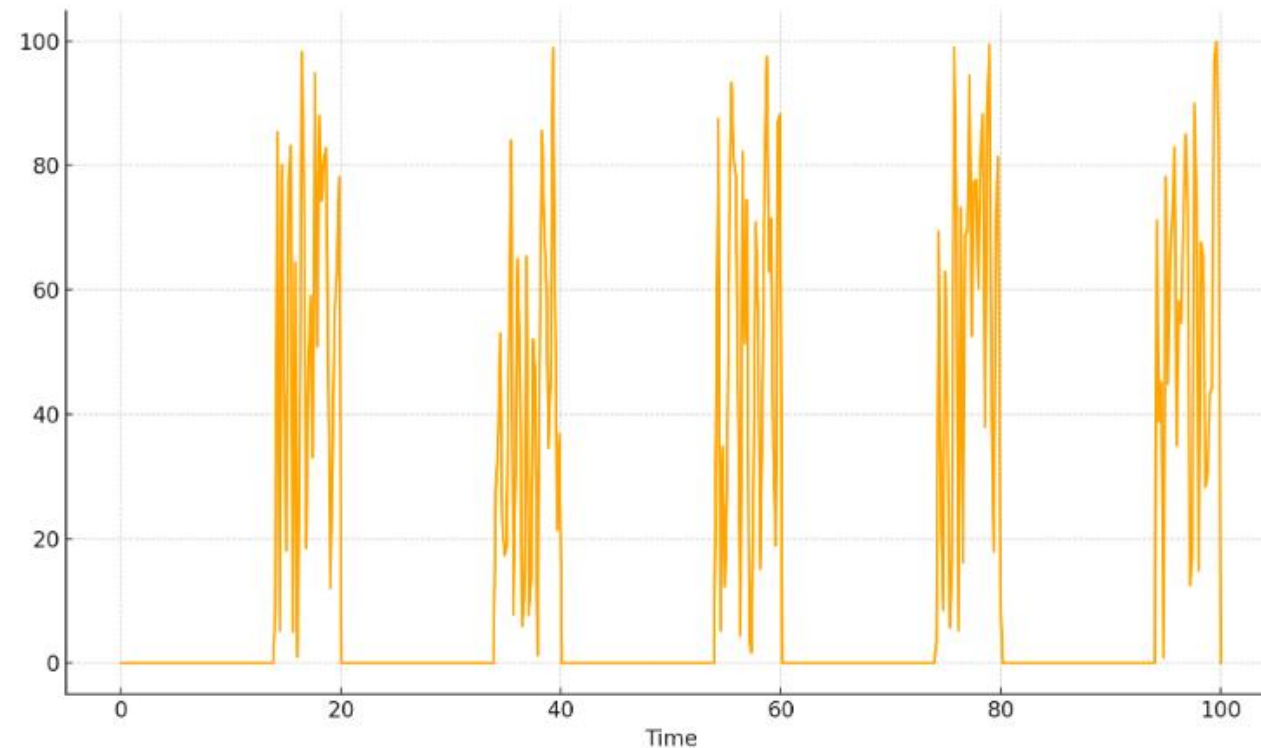
- Sometimes a trend is expected or reasonable
- The graphs shows daily visit for *ChatGPT* and *Instagram*
- The *ChatGPT* traffic shows a clear trend: no traffic in the weekends, but this is not an anomaly

Daily visits: ChatGPT vs. Instagram



Example of pattern in modern datacenters

- In the context of collective communication (AI clusters) bursts are the normal traffic



Traffic Modeling



How can I model traffic?

- Time series
 - Holt-Winters Exponential Smoothing
 - ARIMA (AutoRegressive Integrated Moving Average)
 - capture seasonal trends AND work best for stationary patterns
- Signal processing techniques like Wavelet-based Models
 - match highly-variable patterns but require careful tuning
- Neural Networks, e.g., LSTM, RNN, LLM – not covered in this course
 - perform well on complex trends



Introduction to Time Series Models

- **Time series** models are designed to forecast future values based on past observations. They capture temporal dependencies, trends, and seasonality in the data.
- They often employ a forecasting technique called minimization of the Mean Squared Forecast Error (MSFE) to predict future values of the series. The MSFE metric takes into account the cumulative errors encountered so far.
- Typical setting: one-step-ahead prediction, which focuses on **forecasting** the next time step, denoted as y_{t+1} based on the latest observation at time t , $y_t, y_{t-1}, y_{t-2}, \dots$ where y_t represents the observed value at time t



Methodology step by step

1. Data are **collected, aggregated and filtered**. The size of the aggregation interval influences anomaly detection procedures, mainly the training speed of the detection model.
2. **Anonymization**, e.g., randomly assigning numbers as identifiers for IP addresses, institutions, and institution subnets during database creation.
3. Data preparation: **normalization** and **stationarity** checks.
4. Model **training**, fitting the model to observed data.
5. **Forecasting**, from 1 to up to 10-steps ahead.
6. **Evaluation** using error metrics, e.g., RMSE, MAE, SMAPE.

Clean time series data

In order to be able to perform an adequate analysis, it is important to carefully process and clean data. The first step to achieving this goal is to check data for **missing values**.

In *pandas*, missing values in a DataFrame can be found with the `.isnull()` method.

To replace the missing values in the data, we can use different options such as the mean value, the value from the preceding time point, or the value from time points that are coming after.

We can use the `.fillna()` method in *pandas*, where the function argument specifies how we want to deal with our missing data, e.g.,

- **bfill** (i.e., backfilling) missing values are replaced by the next valid observation
- **ffill** (i.e., forward filling) will replace the missing values with the most recent non-missing value

Introduction to stationarity

Stationary means that the distribution of the data doesn't change with time. For a time series to be stationary, it must fulfill three criteria:

- The series has zero trends. It isn't growing or shrinking.
- The variance is constant. The average distance of the data points from the zero line isn't changing.
- The autocorrelation is constant. How each value in the time series is related to its neighbors stays the same

For time series models, data must be stationary. The reason for this is that modeling is all about estimating parameters that represent the data; therefore, if the parameters of the data are changing with time, it will be difficult to estimate all the parameters.

Check stationarity in python

One way to check stationarity is with eyes.

But the most used method for identifying whether a time series is non-stationary is the **augmented Dicky-Fuller test**. This is a statistical test, where the null hypothesis is that the time series is non-stationary due to trends.

```
from statsmodels.tsa.stattools import adfuller

results = adfuller(data)
print(results)
```

The results object is a tuple:

- **test statistic value**, the more negative this number is, the more likely that the data is stationary.
- **p-value**, if the p-value is smaller than 0.05, we reject the null hypothesis and assume our time series must be stationary.



What to do with non-stationary data?

We need to transform the data into a stationary form before we can model it.

We can think of this a bit like feature engineering in classic machine learning.

One very common way to make a time series stationary is to take its difference. This is where from each value in our time series we subtract the previous value.

```
# Calculate the first difference and drop the nans
data_diff = data.diff()
data_diff = data_diff.dropna()

# Run test and print
result_diff = adfuller(data_diff)
print(data_diff)
```

Sometimes we need to perform other transformations to make the time series stationary. This could be to take the log, or the square root of a time series, or to calculate the proportional change.

AutoRegressive Integrated Moving Average



ARIMA

- ARIMA stands for AutoRegressive Integrated Moving Average. ARIMA models are commonly used to represent stationary time series in various domains where a single variable is measured at regular intervals.
- These models are particularly well-suited for applications in econometrics and financial markets but have also been extensively explored in other fields since the 1970s. In our context, they are useful for anomaly detection, load prediction, and pattern recognition in network traffic.
- Combines **autoregression** (AR), **differencing** (I), and **moving average** (MA). It requires tuning of parameters (p, d, q) for optimal performance.

ARIMA (cont'd)

- **Autoregression (AR):** refers to a model that shows a changing variable that regresses on its own *lagged*, or prior, *values*.
 - A simple AR model is: $y(t) = a(1) * y(t - 1) + \epsilon(t)$
 - $a(1)$ is the *autoregressive coefficient* at lag one, i.e., the slope of the line for a linear regression
 - $\epsilon(t)$ is the *noise* or *shock term*, i.e., white noise, meaning each shock is random and not related to the other shocks in the series
- **Integrated (I):** represents the differencing of raw observations to allow the time series to become stationary (i.e., data values are replaced by the difference between the data values and the previous values)

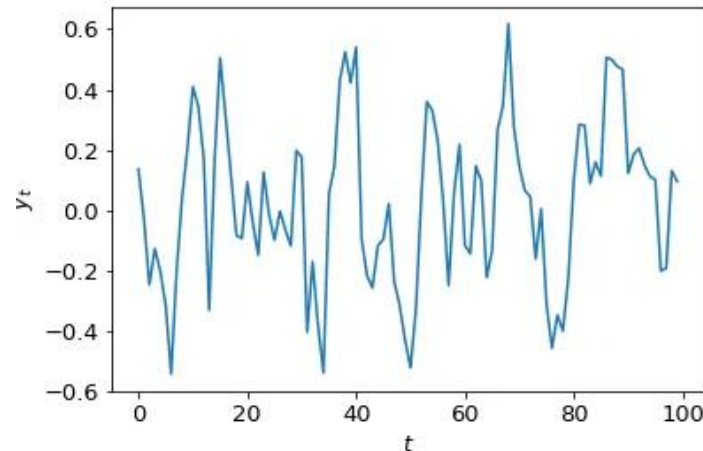


ARIMA (cont'd)

- **Moving average (MA):** incorporates the dependency between an observation and a residual error from a moving average model applied to lagged observations.
 - A simple first-order MA model is: $y(t) = m(1) * \epsilon(t-1) + \epsilon(t)$

We can generate ARIMA-like data,

e.g., $y(t) = 0.5 y(t-1) + 0.2 \epsilon(t-1) + \epsilon(t)$ looks like:



ARIMA (cont'd)

Each component in ARIMA functions as a parameter with a standard notation. For ARIMA models, a standard notation would be ARIMA with p , d , and q , where integer values substitute for the parameters to indicate the type of ARIMA model used. The parameters can be defined as:

- p : the number of lag observations in the model, also known as the lag order. It represents the order of the AR part of ARIMA
- d : the number of times the raw observations are differenced; also known as the degree of differencing and the I part.
- q : the size of the moving average window, also known as the order of the MA part.

d is always an integer, while p and q may either be integers or lists of integers.



ARIMA (cont'd)

The model is often denoted as $ARIMA(p,d,q)$.

A value of zero (0) means that particular component should not be used in the model. This way, the ARIMA model can be constructed to perform the function of an ARMA model, or even simple AR, I, or MA models.

For example, to fit an AR model, we can simply use the ARMA class with q equal to zero; to fit an MA model, we set p equal to zero.

Train-test split

- Generally, in machine learning, we have a *training set* on which to fit the model, and a *test set* on which to test the predictions against. Time series forecasting is just the same, but train-test split is typically done using dates.
- We use the past values to make future predictions, and so we will need to split the data in time. We train on the data earlier in the time series and test on the data that comes later.

Determining ARIMA parameters

- To determine the most appropriate p, d, q , we can use the ACF plot and PACF plots.
- A rough rubric for when to use **I** terms in the model is based on number of times to differentiate to make the data stationary (see DF test).
- A rough rubric for when to use **AR** terms in the model is when:
 - ACF plots show autocorrelation decaying towards zero
 - PACF plot cuts off quickly towards zero
 - ACF of a stationary series shows positive at Lag – 1
- A rough rubric for when to use **MA** terms in the model is when:
 - Negatively Autocorrelated at Lag – 1
 - ACF that drops sharply after a few lags
 - PACF decreases gradually rather than suddenly

Implementing ARIMA step-by-step

1. Import Libraries

First, import the necessary libraries for time series analysis.

e.g., numpy, pandas, statsmodel, sklearn

2. Load Dataset

Let's download/load and use the dataset of interest.

3. Extracting data of interest

With the desired granularity, daily, weekly, monthly data.

Implementing ARIMA step-by-step (cont'd)

4. Plotting the Obtained Data

Just to have an idea of the measurements. We can get a rough idea of seasonality, patterns, trends.

5. Check Stationarity

Before applying ARIMA, check if your time series data is stationary. **Stationarity** refers to the statistical properties of a time series remaining constant over time, such as constant mean, constant variance, and constant autocovariance. We can use the Dickey-Fuller (DF) test for this.

```
def check_stationarity(timeseries):  
    # Perform the Dickey-Fuller test  
    result = adfuller(timeseries, autolag='AIC')  
    p_value = result[1]  
    print(f'ADF Statistic: {result[0]}')  
    print(f'p-value: {p_value}')  
    print('Stationary' if p_value < 0.05 else 'Non-Stationary')
```

Implementing ARIMA step-by-step (cont'd)

6. Identify Model Parameters

Identify the ARIMA model parameters (p , d , q) using ACF and PACF plots.

ACF Plot: a plot of autocorrelations of the differenced time series. **Autocorrelation** measures the relationship between a data point and previous data points at different lags. So $ACF(1)$ is the correlation between the time series and a one-step lagged version of itself, $ACF(2)$ is the correlation with the two-step lagged version of itself, and so on.

Implementing ARIMA step-by-step (cont'd)

PACF Plot: a plot of partial autocorrelations of the differenced time series. **Partial autocorrelation** represents the correlation between a data point and a lag after subtracting the effect of correlation at smaller lags.

```
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.graphics.tsaplots import plot_pacf

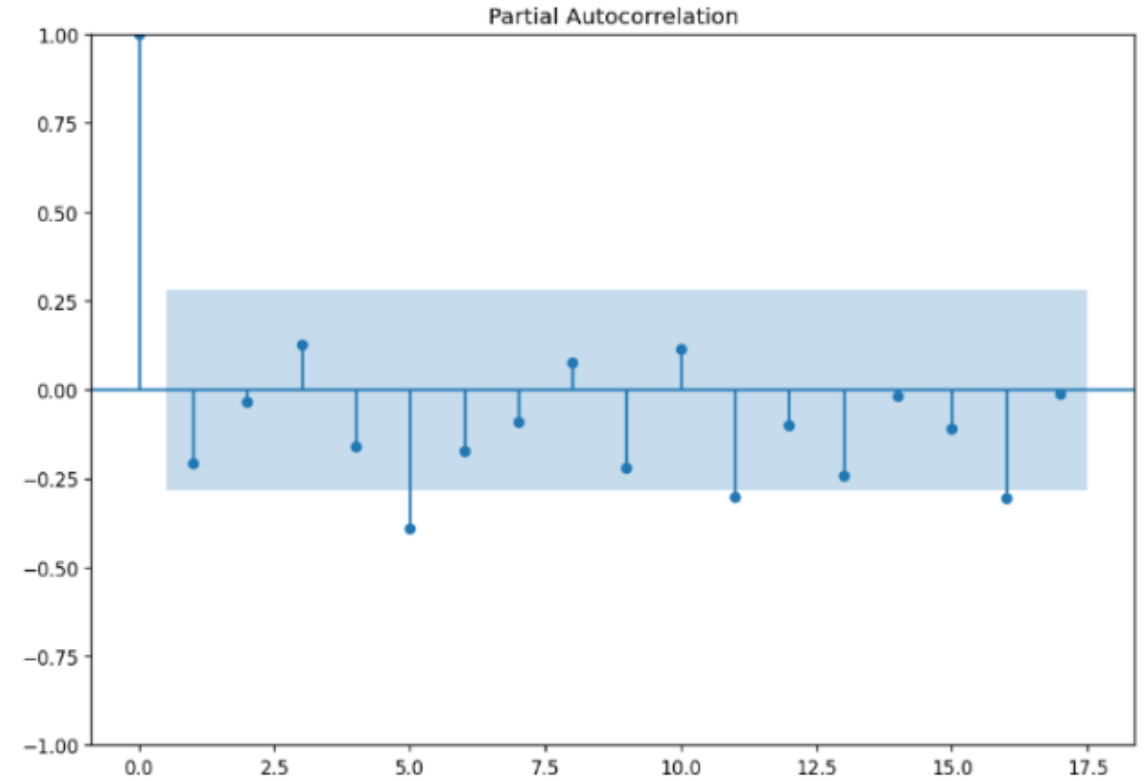
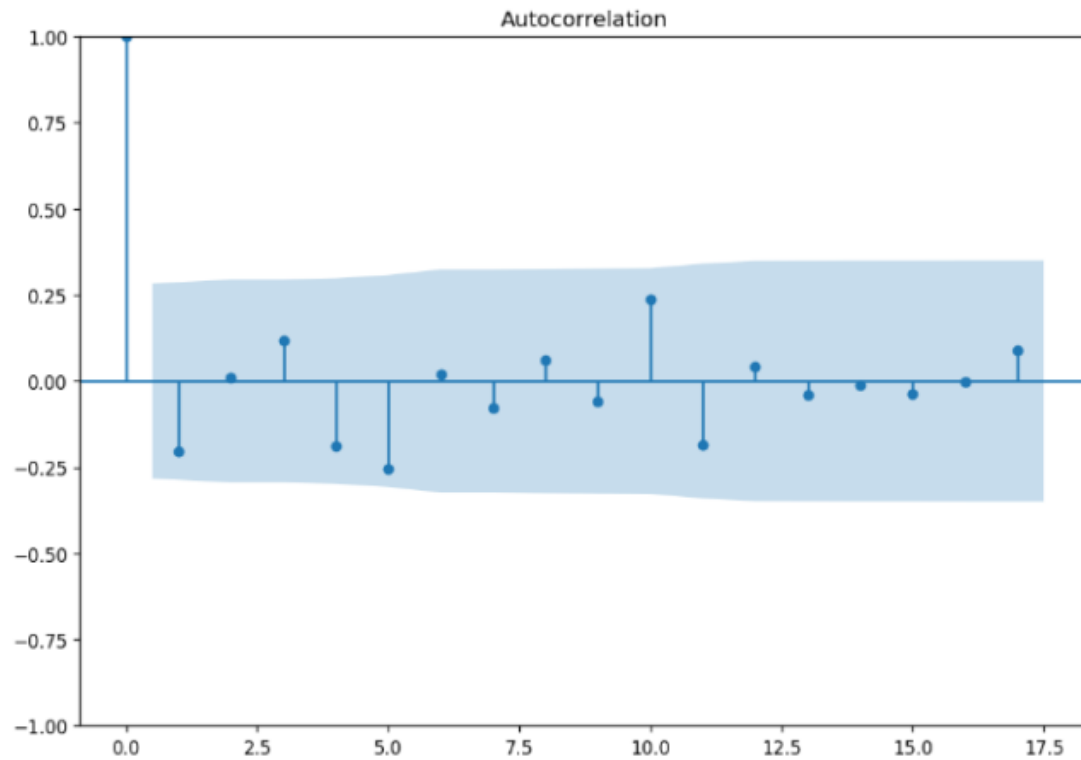
# plot ACF and PACF

plot_acf(data)
plot_pacf(data)
```

Remarks. The time series must be made stationary before making these plots. If the ACF values are high and tail off very slowly, therefore this is a sign that the data is non-stationarity, and it needs to be different. On the other hand, if the autocorrelation at lag-1 is very negative this is a sign that we have taken the difference too many times.



ACF and PACF examples



Both plots have on the x-axis the lags, and the y-axis the correlation coefficients ranging from -1 to 1 where -1 means perfect negative correlation, 0 means no correlation and 1 means perfect positive correlation.

Interpreting ACF, PACF

There is a **blue area** in the ACF and PACF plots, which depicts the 95% confidence interval and is an indicator for the significance threshold. That means, anything within the blue area is statistically close to zero and anything outside the blue area is statistically non-zero.

p: Significant at each lag p / Cuts off after lag p in *PACF*

q: Significant at each lag q / Cuts off after lag q in *ACF*

Rule of thumb:

- The number of lags before the cutoff gives the order
- If neither cuts off clearly → try combinations and choose based on Akaike information criterion (AIC) and Bayesian information criterion (BIC), also recommended for a more formal finding of these values



Implementing ARIMA step-by-step (cont'd)

7. Fit the ARIM model

Now, fit the ARIMA model using the identified parameters. The ARIMA function takes the data and the specified order parameters. The fit method is then called to estimate the model's coefficients based on the data.

8. Generate forecasts

With the model fitted, generate forecasts for future time periods. The *results.get_forecast(steps=forecast_periods)* function generates the forecast, providing both the predicted mean and a confidence interval for the forecasted values.

```
# Forecast future values
forecast_periods = 12 # Forecast the next 12 months
forecast = results.get_forecast(steps=forecast_periods)
forecast_mean = forecast.predicted_mean
forecast_ci = forecast.conf_int()
```

```
# Plot the forecast
```

Implementing ARIMA step-by-step (cont'd)

9. Evaluate the model

Lastly, we can evaluate the forecasted values by comparing them to the observed sales data using three common metrics for this evaluation: Root Mean Squared Error (RMSE), Symmetric Mean Absolute Percentage Error (SMAPE), and R-squared (R^2).

For **RMSE** and **SMAPE**, lower values indicate better performance.

The R^2 value ranges from 0 to 1, with 1 indicating that the model perfectly predicts the data and 0 indicating that the model does not explain any of the variance in the data.

Conclusion

- ARIMA is a valid approach to **forecast network traffic** in the future
 - You'll practice it in the lab, also for a variant of ARIMA as SARIMA
- Simple models work
 - We don't need LLMs, e.g., ChatGPT and Claude, for everything
- Predicting network traffic is helpful to detect anomalies, attacks, or simply to better manage the infrastructure
 - Although mastering these methods is complex and require some domain knowledge